



Duale Hochschule Baden-Württemberg Mannheim

Projektarbeit

Rekurrente Neuronale Netze

Studiengang Wirtschaftsinformatik

Studienrichtung Data Science und Künstliche Intelligenz

Verfasser(in):	Luke Vierfuß und Sebastian Höfling
Matrikelnummer:	1091660 und 6043131
Firma:	Fresenius SE & Co. KGaA. und DATEV eG
Abteilung:	-
Kurs:	WDSKI25A
Studiengangsleiter:	Prof. Dr.-Ing. Pfisterer
Wissenschaftliche(r) Betreuer(in):	-
Firmenbetreuer(in):	-
Bearbeitungszeitraum:	04.05.2026 – 23.05.2026

Ehrenwörtliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Titel "*Rekurrente Neuronale Netze*" selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum

Luke Vierfuß und Sebastian Höfling

Sperrvermerk

Ein Sperrvermerk sollte nur bei berechtigtem Bedarf gesetzt werden!

Beachten Sie, dass mit Sperrvermerk versehene Arbeiten nicht für weitere wissenschaftliche Zwecke außerhalb des Firmenkontextes oder zur Publikation verwendet werden dürfen.

Wir empfehlen, wenn möglich, auf den Sperrvermerk zu verzichten.

Besprechen Sie diese Problematik mit Ihrer Firma!

(Mustertext) Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte vorliegt.

Danksagung

Hier können Sie eine Danksagung schreiben.

Contents

List of Figures	v
List of Tables	vi
Quelltextverzeichnis	vii
Abkürzungsverzeichnis	ix
Kurzfassung (Abstract)	x
1 Motivation und Zielsetzung	1
1.1 Motivation	1
1.2 Zielsetzung	1
2 Voraussetzungen	2
2.1 Grundlagen der Mathematik	2
2.1.1 Notation	2
2.1.2 Partielle Ableitung	2
2.1.3 Kettenregel	2
2.1.4 Gradient	3
2.1.5 Jacobi-Matrix	3
2.2 Grundlagen des Maschinellen Lernens	3
2.2.1 Datenvorbereitung	3
2.2.2 Regression	4
2.2.3 Neuronale Netze	4
2.2.4 Backpropagation	5
3 Zusammenfassung	6
3.1 Fazit	6
3.2 Ausblick	6
Anhang	
Bibliography	7

List of Figures

List of Tables

2.1	Im Dokument verwendete mathematische Notation	2
-----	---	---

Quelltextverzeichnis

Algorithmenverzeichnis

Abkürzungsverzeichnis

AD	Archiv für Diplomatik, Schriftgeschichte, Siegel- und Wappenkunde
BMBF	Bundesministerium für Bildung und Forschung
DHBW	Duale Hochschule Baden-Württemberg
ECU	European Currency Unit
EU	Europäische Union
RDBMS	Relational Database Management System
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
BPTT	Backpropagation Through Time

Kurzfassung (Abstract)

Hier können Sie die Kurzfassung (engl. Abstract) der Arbeit schreiben. Beachten Sie dabei die Hinweise zum Verfassen der Kurzfassung.

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

1 Motivation und Zielsetzung

1.1 Motivation

//TODO: Warum RNNs? Warum ist das Thema relevant? Limitierung der normalen Feedforward-Netze bei sequenziellen Daten, z.B. Zeitreihen

Trotz ihrer theoretischen Stärke leiden einfache Recurrent Neural Networks (RNNs) in der Praxis unter dem Problem verschwindender und explodierender Gradienten, sobald lange Abhängigkeitsketten erlernt werden müssen. Die Entwicklung von *Long Short-Term Memory*-Einheiten (LSTM) und *Gated Recurrent Units* (GRU) hat dieses Problem weitgehend gelöst und den Einsatzbereich rekurrenter Architekturen erheblich erweitert.

//TODO: Praxisbeispiele nennen, z.B. Sprachmodellierung, Zeitreihenanalyse, etc.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Projektarbeit ist es, RNNs als Klasse sequenzieller Lernmodelle theoretisch fundiert aufzuarbeiten und ihre praktischen Eigenschaften anhand von konkreten Beispielen zu veranschaulichen.

Hieraus leiten sich die folgenden Teilziele ab:

1. **Theoretische Grundlagen:** Darstellung des mathematischen Modells einfacher rekurrenter Netze, einschließlich der Vorwärtsberechnung und der zeitlich entfalteten Rückwärtspropagation (*Backpropagation Through Time*, BPTT).
2. **Gradientenproblematik:** Analyse der Ursachen verschwindender und explodierender Gradienten sowie Diskussion der wichtigsten Gegenmaßnahmen.
3. **Erweiterte Architekturen:** Vorstellung der GRU-Architekturen als Lösungsansatz für das Kurzzeitgedächtnis-Problem einfacher RNNs.
4. **Anwendungsbeispiele:** Prototypische Implementierung und Evaluation eines RNN-basierten Modells an einem repräsentativen Datensatz zum Thema Aktienpreisvorhersage, um die erarbeiteten Konzepte zu veranschaulichen.
5. **Kritische Einordnung:** Reflexion der Grenzen und Stärken rekurrenter Netze im Vergleich zu modernen Alternativen wie *Transformer*-Architekturen.

2 Voraussetzungen

2.1 Grundlagen der Mathematik

2.1.1 Notation

Symbol	Bedeutung
x	Skalar (kursiver Kleinbuchstabe)
$\mathbf{x}$	Vektor (fetter Kleinbuchstabe, Spaltenvektor)
$\mathbf{x}^\top$	Transponierter Zeilenvektor
$\mathbf{W}$	Matrix (fetter Großbuchstabe)
W_{ij}	Element in Zeile i , Spalte j von $\mathbf{W}$
$\mathbb{R}$	Menge der reellen Zahlen
$\mathbb{R}^n$	n -dimensionaler reeller Vektorraum

Table 2.1: Im Dokument verwendete mathematische Notation

2.1.2 Partielle Ableitung

Gegeben eine Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$, die von mehreren Variablen $x_1, x_2, \dots, x_n$ abhängt, beschreibt die partielle Ableitung

$$\frac{\partial f}{\partial x_i} \quad (2.1)$$

die Änderungsrate von f bezüglich x_i , während alle anderen Variablen konstant gehalten werden. Partielle Ableitungen sind die Grundlage der mehrdimensionalen Differentialrechnung und bilden das mathematische Fundament für Optimierungsverfahren im maschinellen Lernen.

2.1.3 Kettenregel

Die Kettenregel beschreibt, wie die Ableitung einer zusammengesetzten Funktion berechnet wird. Sind f und g differenzierbare Funktionen mit $h(x) = f(g(x))$, so gilt:

$$\frac{dh}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx} \quad (2.2)$$

2.1.4 Gradient

Der Gradient einer skalaren Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ist der Vektor aller partiellen Ableitungen:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \quad (2.3)$$

Er zeigt in die Richtung des steilsten Anstiegs von f am Punkt $\mathbf{x}$.

2.1.5 Jacobi-Matrix

Die Jacobi-Matrix verallgemeinert den Gradienten auf vektorwertige Funktionen. Gegeben eine Funktion $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, die einen Eingangsvektor $\mathbf{x} \in \mathbb{R}^n$ auf einen Ausgangsvektor $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^m$ abbildet, ist die Jacobi-Matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ definiert als:

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}. \quad (2.4)$$

Das Element $J_{ij} = \frac{\partial f_i}{\partial x_j}$ gibt an, wie stark sich die i -te Ausgabekomponente ändert, wenn die j -te Eingabekomponente variiert wird.

Für den Spezialfall $m = 1$ reduziert sich die Jacobi-Matrix zum transponierten Gradienten: $\mathbf{J} = \nabla f^\top$. Im Kontext neuronaler Netze tritt die Jacobi-Matrix bei der Anwendung der Kettenregel auf zusammengesetzte vektorwertige Funktionen auf und beschreibt die lineare Näherung einer Schicht an einem gegebenen Punkt.

2.2 Grundlagen des Maschinellen Lernens

2.2.1 Datenvorbereitung

Bevor ein Modell trainiert werden kann, müssen die Rohdaten in eine geeignete Form gebracht werden. Dieser Schritt wird als Datenvorbereitung bezeichnet und umfasst mehrere Teilaufgaben.

Zunächst werden fehlende Werte behandelt, etwa durch Entfernen betroffener Einträge oder durch das Einsetzen von statistischen Kennzahlen wie dem Mittelwert. Anschließend erfolgt

häufig eine Normalisierung oder Standardisierung der Eingabemerkmale, damit numerisch sehr unterschiedliche Größen den Trainingsprozess nicht verschlechtern. Bei der Standardisierung wird jedes Merkmal x transformiert zu

$$x' = \frac{x - \mu}{\sigma}, \quad (2.5)$$

wobei μ der Mittelwert und σ die Standardabweichung des jeweiligen Merkmals sind.

Schließlich wird der Datensatz in mindestens zwei disjunkte Teilmengen aufgeteilt: einen Trainings- und einen Testdatensatz. Optional wird ein dritter Validierungsdatensatz abgetrennt, der zur Hyperparameterauswahl während des Trainings genutzt wird.

2.2.2 Regression

Regression bezeichnet statistische Methoden zur Modellierung der Beziehung zwischen einer Zielgröße und einer oder mehreren unabhängigen Merkmalen. Das einfachste Modell ist die lineare Regression, die eine lineare Abbildung von der Eingabe $\mathbf{x} \in \mathbb{R}^n$ auf eine vorhergesagte Ausgabe $\hat{y} \in \mathbb{R}$ annimmt:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad (2.6)$$

wobei $\mathbf{w}$ der Gewichtsvektor und b der Bias-Term sind.

Die Qualität der Vorhersage wird durch eine Verlustfunktion $\mathcal{L}$ gemessen. Das Ziel des Lernalgorithmus ist es, die Parameter $\mathbf{w}$ und b so zu wählen, dass $\mathcal{L}$ minimiert wird.

2.2.3 Neuronale Netze

Ein künstliches neuronales Netz besteht aus mehreren hintereinandergeschalteten Schichten von sog. Neuronen. Jedes Neuron berechnet eine gewichtete Summe seiner Eingaben und wendet darauf eine nichtlineare Aktivierungsfunktion σ an:

$$a = \sigma(\mathbf{w}^\top \mathbf{x} + b). \quad (2.7)$$

Eine Schicht transformiert einen Eingangsvektor $\mathbf{x}$ zu einem Ausgangsvektor $\mathbf{a}$ mittels einer Gewichtsmatrix $\mathbf{W}$ und eines Biasvektors $\mathbf{b}$:

$$\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}). \quad (2.8)$$

Durch das Hintereinanderschalten mehrerer solcher Schichten entsteht ein *tiefes* Netz (*Deep Neural Network*), das in der Lage ist, hochgradig nichtlineare Zusammenhänge zu erlernen.

Das gesamte Netz realisiert damit eine parametrisierte Funktion $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$, deren Parameter $\theta = \{\mathbf{W}^*, \mathbf{b}^*\}$ durch Training bestimmt werden.

2.2.4 Backpropagation

Der Algorithmus zur Berechnung der Gradienten in neuronalen Netzen wird als Backpropagation bezeichnet. Er wendet die Kettenregel der Differentialrechnung rekursiv an, um die partiellen Ableitungen der Verlustfunktion $\mathcal{L}$ nach allen Netzwerkparametern effizient zu berechnen.

Sei $\mathbf{a}^{(l)}$ die Aktivierung der l -ten Schicht und $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ die zugehörige Voraktivierung. Der sogenannte Fehlerterm (Delta) der Schicht l ist definiert als

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \quad (2.9)$$

Mithilfe der Kettenregel lässt er sich aus dem Fehlerterm der nachfolgenden Schicht $l + 1$ rückwärts propagieren:

$$\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)}), \quad (2.10)$$

wobei $\odot$ das elementweise Produkt bezeichnet. Die Gradienten nach den Gewichten ergeben sich dann zu $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top$.

In der Praxis wird Backpropagation mit einem Gradientenabstiegsverfahren kombiniert. Beim stochastischen Gradientenabstieg (*Stochastic Gradient Descent*, SGD) werden die Parameter nach jedem Mini-Batch aktualisiert:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \quad (2.11)$$

wobei $\eta > 0$ die Lernrate ist. // TODO: Fact-Check Formeln

3 Zusammenfassung

Dieses Kapitel enthält die Zusammenfassung der Arbeit mit Fazit und Ausblick.

3.1 Fazit

...

3.2 Ausblick

...

Bibliography

- [1] N. Andruskiewitsch and J. Devoto. “Extensions of Hopf algebras”. In: *St. Petersburg Math. J.* 7.1 (1996), pp. 17–52.
- [2] Detlef Borchers. *Elektronische Unterschrift: Bundesregierung reformiert Schriftformerfordernis*. Jan. 2, 2015. URL: <http://m.heise.de/newsticker/meldung/Elektronische-Unterschrift-Bundesregierung-reformiert-Schriftformerfordernis-3259140.html> (visited on 01/01/2016).
- [3] *DIN 69901-2. Prozesse, Prozessmodell*. Deutsches Institut für Normung, 2010.
- [4] Erika Musterfrau and Max Exempelmann. *Zitationen am Beispiel*. 5th ed. Wiesbaden: Wissenschaftstestverlag, 2012.
- [5] NN. *Mögliche Stile: Numerisch, Alphabetisch, Harvard, Autor-Jahr*.
- [6] Heribert Testmann, Marita Demofrau, and Eva Checker. “Das Testen von Artikeln”. In: *Int. Journal of Testing*. Lecture Notes 5.2 (42 2015), pp. 111–222.